

摘要：

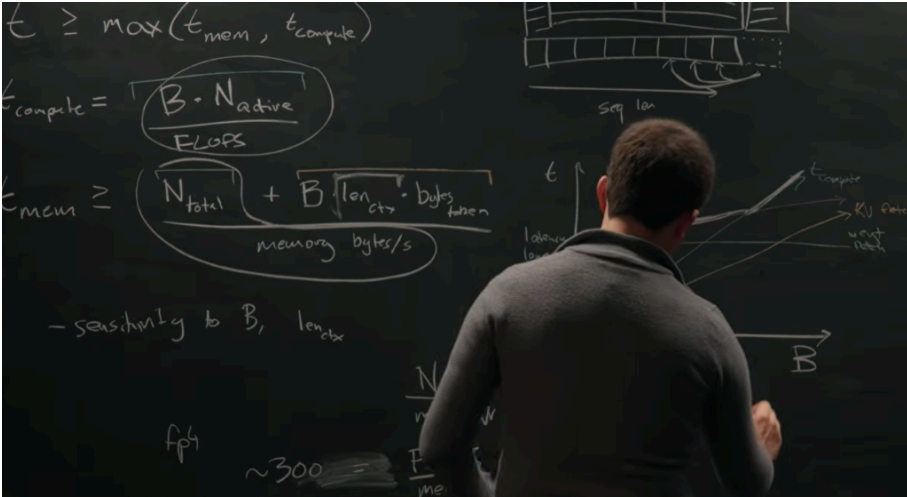
前谷歌TPU架构师Pope完成了一次AI“解密”：他估算，如果不批量处理用户请求，单次推理成本可能高出1000倍。而GPT-5的预训练数据量，是理论最优解的100倍。此外，DeepSeek V3拥有256个专家，每次推理只激活其中一小部分。MoE（混合专家）架构被限制在一个机架72块GPU以内，这是制约模型规模扩展的核心物理瓶颈之一。

一块黑板、几个方程式，芯片工程师Reiner Pope用这些工具，拆解了GPT-5、Claude和Gemini背后的训练与推理逻辑，并从公开的API定价中，反推出大模型不愿公开的架构细节。

近日，知名科技播客主持人Dwarkanish Patel与芯片创业公司MatX的CEO Reiner Pope进行了一场罕见以黑板推演为形式的深度对话。Pope此前在谷歌负责TPU架构与编译器优化，被认为是少数真正贯通AI全栈——从芯片设计到模型架构——的工程师之一。

Pope在黑板前用方程和图表，系统拆解了前沿大模型从训练到推理的底层逻辑。在Dwarkanish看来，这些细节“一旦理解，AI为何是今天这个样子——架构、定价、进步速度——就全都说得通了”。

核心结论包括：如果不批量处理用户请求，单次推理成本可能高出1000倍。而GPT-5的预训练数据量，是理论最优解的100倍。此外，DeepSeek V3拥有256个专家，每次推理只激活其中一小部分（32个）。MoE（混合专家）架构被限制在一个机架72块GPU以内，这是制约模型规模扩展的核心物理瓶颈之一。



一块GPU机架，决定了模型有多大

要理解顶级大模型为何是现在这个样子，得先从硬件说起。

现代大模型推理跑在GPU集群上。英伟达Blackwell NVL72是目前主流的部署形态——一个机架塞了72块GPU，通过NVLink高速互联，任意两块GPU之间只需两跳（经过中间交换机），通信带宽极高。

但一旦跨出这个机架，通信速度就慢了8倍。

这个“8倍差距”，直接决定了MoE（混合专家模型）的部署上限。

DeepSeek V3拥有256个专家，每次推理只激活其中一小部分（32个）。Pope解释，最自然的部署方式是“专家并行”——不同专家放在不同GPU上。任何GPU都可能向任何其他GPU发送token，这是一种“全对全”（all-to-all）通信模式，和机架内NVLink的拓扑结构完美契合。

但一旦专家分布到两个机架，问题就来了：跨机架的token有一半要走慢8倍的网络，直接成为瓶颈。

“一个机架的大小，限制了你能做多大的专家层。” Pope说。

这就解释了一个市场上长期困惑的问题：为什么Gemini看起来比其他实验室更早取得大模型预训练的成功？Pope的推断是，谷歌的TPU系统长期拥有更大的scale-up域，能在更大范围内做全对全通信，这让它可以部署更高稀疏度的MoE模型，同时维持推理效率。



批处理：省1000倍成本的秘密

访谈还提及一个市场常见现象：Claude、Codex等产品提供“快速模式”，价格高出6倍，速度却只快2.5倍。为什么？能不能反过来，用“慢速模式”换取更低价格？

Pope的回答直接：核心变量是批处理规模（batch size）。他用一个“发车时刻表”的比喻解释了背后的逻辑。

GPU每隔约20毫秒发出一班“列车”（执行一次批处理推理）。每班列车能搭多少乘客，就是批处理大小（batch size）。

核心结论是：推理的单位成本，在批处理量小的时候极高，随着批处理增大会急剧下降，最终趋于一个下限。

原因是权重加载成本的摊销。每次推理都要把模型权重从内存（HBM）读入芯片。这个成本是固定的，不管服务1个用户还是2000个用户，权重只读一次。如果只服务1个用户，这个固定成本就全压在他身上；服务2000个用户，成本均摊后几乎可以忽略不计。

Pope估算，如果不做批处理，成本可以高出1000倍。

那最优批处理规模是多少？Pope给出了一个简洁的公式：约等于300乘以模型稀疏度。对DeepSeek这类激活1/8专家的模型，大约是2400个并发序列。这个数字与模型总参数量无关，只取决于硬件特性和稀疏度——这是一个“反直觉”的结论。

所以，“慢速模式”真的能便宜很多吗？从数学上看，不太行。KV缓存（存储每个用户历史对话的内存）无法在不同用户之间共享摊销，因此让用户多等并不能显著降低成本。Pope说：“（慢速模式）节省不了太多，因为KV缓存是每个用户独立的，计算量也是独立的。”

从API定价，反推模型架构

Pope展示了一个让人印象深刻的推理过程：**通过公开的API定价，可以反推出模型的内部架构参数。**

线索一：Gemini在20万 token处涨价50%，为什么恰好是50%？为什么恰好在20万Token这个节点？

Gemini 3.1的定价在超过20万 token后上涨50%。Pope解释，这对应着KV缓存的内存带宽成本超过权重矩阵计算成本的临界点——也就是模型从“计算瓶颈”切换到“内存带宽瓶颈”的转折点。

他进一步用这个数字反算：假设激活参数约1000亿，临界点在20万 token，可以推算出每个token的KV缓存大约占**2KB**。这与Character AI等公开论文中描述的注意力机制参数（8个KV头，维度128）高度吻合。

“他们通过API定价泄露了相当多的信息。” Pope说，“当然，他们有动力把价格定得接近成本，否则竞争对手可以抢走用户。”

线索二：输出比输入贵5倍

大多数模型的输出token（decode）比输入token（prefill）贵约3-5倍。原因在于：

- **Prefill阶段**：一次性并行处理大量输入token，计算效率高，接近“计算瓶颈”
- **Decode阶段**：每次只生成一个token，要读取全部模型权重和KV缓存，极度受内存带宽瓶颈制约

这个价格差，实际上量化了当前顶级模型推理时的内存带宽瓶颈程度。

线索三：缓存命中为何便宜10倍

API通常对“缓存命中”的token大幅打折。Pope解释，这对应的是**存储KV缓存存在不同内存层级的成本差异**：重新计算一次（从token ID从头生成KV缓存）versus从HBM/DDR/闪存中直接读取。

他进一步推算，按照Gemini“5分钟缓存”与“1小时缓存”的定价差异，可以推断这两个档位对应的存储介质分别是**闪存和机械硬盘**——后者让Pope也感到惊讶：“我没想到机械硬盘会被用在这里。”

GPT-5过度训练了多少？答案是100倍

这是整场讲座最具震撼性的推算。

Pope从一个经济学直觉出发：**当预训练成本、RL训练成本、推理成本三者大致相等时，整体效率最优。**

他把这三块成本写出来，发现激活参数量这个变量直接消掉了——也就是说，最优训练量的推算与模型大小本身无关，只取决于推理流量。

然后他代入真实数字：

- 假设某前沿模型推理流量约**5000万token/秒**（全部流量除以一个家族中的多个模型版本）
- 模型生命周期约**2个月**（在下一版本发布前）
- 合计推理token数约**200万亿**（ 2×10^{14} ）

Chinchilla最优解（基于约1000亿激活参数）大约是**2万亿token**。

两者之比：**100倍**。

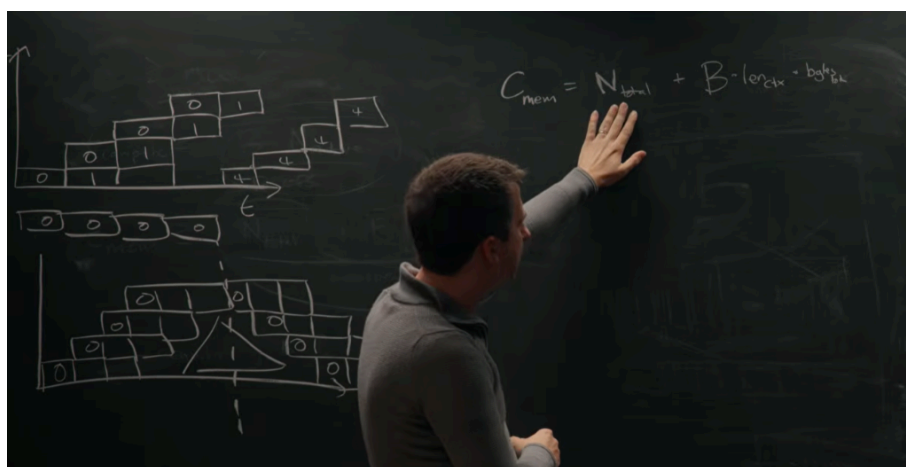
也就是说，当前顶级模型的预训练数据量，约是从纯训练效率角度出发所需数据量的100倍。

"我们知道这大概是对的，因为有传言说GPT-5预训练了约150万亿token，和我们算出的200万亿很接近。" Patel说。

Pope补充说，这个推算的核心逻辑是：**你花在服务用户上的计算，应该和你花在训练上的计算大体相当。否则，就是在某一头浪费钱。**

用Patel的话说："如果GPT-5要被最优地训练，那么所有用户使用它产生的token总量，应该等于预训练消耗的token总量——而预训练数据，大约就是人类知识的总和。"

Pope对此回应："大致如此。"



流水线并行：听起来很美，但大多数时候用不上

关于流水线并行（把模型的不同层分散到不同机架上串行执行），Pope的结论是：**它能节省内存容量，但解决不了KV缓存问题，因此在推理场景价值有限。**

直觉上，流水线并行需要同时保持多个"在途"的batch，这让全局batch大小随流水线级数成比例增长。虽然每个机架上的权重存储减少了，但所有机架上的KV缓存总量并没有减少——因为需要更多并发序列来填满流水线。

"你无法跨pipeline阶段摊销KV缓存，就像你无法跨batch摊销KV缓存一样。" Pope总结道。

这也解释了为什么Ilya Sutskever曾说"现在我们都知 道，流水线并行是不明智的"——这句话在访谈中被Patel引用，而Pope的推演给出了工程层面的注解。

神经网络与密码学的“趋同进化”

访谈最后，Pope谈到了他写过的一篇博客观点：神经网络的架构与密码学协议之间存在"趋同进化"。

两者都需要把输入信息在整个系统中充分混合——密码学是为了让输出看起来像随机噪声，神经网络是为了提取隐藏的高层结构。但目标恰好相反：密码学努力破坏结构，神经网络努力发现结构。

Pope提到了一个具体的技术迁移案例：**Feistel网络**——一种密码学中用于让不可逆函数变得可逆的构造，在2017年被引入神经网络，形成了"RevNets"（可逆网络）。RevNets允许在训练的反向传播过程中，无需预先存储所有层的激活值，而是边反向传播边重新计算——用更多计算换取更少内存。

这与KV缓存的逻辑恰好相反：KV缓存是用更多内存换取更少计算。Pope说，"用内存换计算，在当前的硬件条件下通常是合算的。"

访谈全文如下：

GPT-5、Claude 和 Gemini 的训练与推理机制——Reiner Pope 主讲

主持人：Dwarkesh Patel 嘉宾：Reiner Pope（MatX 首席执行官）

节目说明： 本期采用了全新的黑板讲座形式，由 Reiner Pope 系统讲解前沿大语言模型的训练与推理原理。内容涉及大量数据与数学推导，令人惊讶的是，仅凭几个公式、公开的 API 价格和一支粉笔，就能推断出各大实验室正在做什么。内容略有技术性，但非常值得深入了解。

Reiner 是芯片创业公司 MatX 的 CEO（披露：主持人 Dwarkesh 是 MatX 的天使投资人）。他此前在 Google 从事软件效率、编译器和 TPU 架构工作，是极少数能够贯通从芯片设计到模型架构整个技术栈的专家之一。





第一章：批量大小如何影响 Token 成本与速度

Dwarkesh: 今天我采访的是 Reiner Pope，他是新芯片创业公司 MatX 的 CEO。此前他在 Google 主导了 TPU 架构等多项工作。本期采用黑板讲座的全新形式，我们专门为此打造了新的录制空间。今天要聊的话题涵盖模型架构、机器学习基础设施等诸多方面。

我认为这个话题非常重要。一旦你理解了训练和推理在集群中的运作方式，很多问题就会豁然开朗——为什么 AI 是现在这个样子，为什么 AI 架构是现在这个样子，为什么 API 价格是现在这个样子，以及为什么 AI 进步是现在这个节奏。要真正理解这些，你需要深入细节，而深入细节就需要一块黑板。Reiner，非常感谢你来参加。

首先，我想请你解释一个现象。现在有几家公司，比如 Claude、Codex 和 Cursor，都提供类似“快速模式”的选项——花费 6 倍的价格，可以获得 2.5 倍的 Token 输出速度。我有几个问题：

- 这背后的机制是什么？为什么付更多的钱就能获得更低的延迟？
- 这种模式能一直延伸下去吗？比如付 100 倍的价格，能获得更快的速度吗？
- 反过来是否也成立？比如推出“慢速模式”——如果用户愿意等几分钟，能否获得更低廉的价格？

Reiner: 直接说结论：最大的影响因素是**批量大小 (batch size)**。接下来我们会精确量化这一点，分析它对延迟和成本的影响。另外还有一个效应，叫做推测解码 (speculative decoding) 或多 Token 预测 (multi-token prediction)，我们之后可以回头讨论，但首先要讲的是批量大小。

我想引入两个分析原则：

第一，屋顶线分析 (roofline analysis)。我们来分析如何在一个芯片集群上运行 Transformer 模型。以 Blackwell NVL72 集群为例，也就是一个 72 块 GPU 的机架。屋顶线分析关注的是内存带宽和计算性能这两个维度。

第二，只关注模型的两个简单因素：操作权重的时间，以及操作上下文（即 KV 缓存）的时间。

我们尝试估算运行某种形状的推理所需的时间。这不是精确预测，而是近似——我们会说“时间大于等于某个量”。我们考虑两个方面：内存读取所需时间，以及计算所需时间。这个简单模型能给我们非常强的预测能力。

计算时间（t_compute）如何估算？

需要做两件事：一是乘以所有活跃参数；二是做注意力计算。

对于权重矩阵乘法的计算时间，公式如下：

tcompute=B×NactiveFLOPstcompute=FLOPsB×Nactive

【注：B 为批量大小，N_active 为活跃参数数量，FLOPs 为芯片的浮点运算吞吐量。注意力计算部分相对较小，可忽略。】

内存时间（t_mem）如何估算？

需要取出所有权重，以及读取 KV 缓存：

tmem=Ntotal内存带宽+B×Lcontext×bytes_per_token内存带宽tmem=内存带宽Ntotal+内存带宽B×Lcontext×bytes_per_token

【注：N_total 为总参数量（不只是活跃参数），第二项是 KV 缓存读取时间，与批量大小和上下文长度成正比。】

Dwarkesh： 批量指的是同时服务多个用户，对吧？

Reiner： 对。批量的意义也正在于此——如果不把多个用户合并成一批，成本和经济性可能比合并处理差一千倍。我们稍后会清楚地看到这一点。

以 DeepSeek V3 为例，它有约 370 亿活跃参数，总参数约 7000 亿。我们关注的是处理单个 Token 时用到的活跃参数。

关于 KV 缓存，简单解释一下：

在自回归推理的解码阶段，已有一批文本 Token，模型要生成下一个 Token。这一步需要对模型中所有层的权重矩阵做完整的前向传播，同时通过注意力机制，让当前 Token 关注所有历史 Token——它关注的是模型对历史 Token 生成的内部表示，这就是 KV 缓存。

这个"单 Token 关注全部历史"的过程主要由内存读取主导，而非矩阵乘法。因此，内存读取时间由以下公式给出：

tmem=Ntotal+B×Lcontext×bytes_per_token内存带宽tmem=内存带宽Ntotal+B×Lcontext×bytes_per_token

而总时间为：

t=max(tcompute, tmem)t=max(tcompute, tmem)

批量大小 vs. 延迟（latency）图像分析：

我们先画批量大小与时间的关系图。



- **t_compute**（计算时间）：与批量大小线性正比，无偏移量，是一条过原点的直线。
- **t_mem**（内存时间）：由两部分组成。
 - 权重读取：是一个与批量大小无关的常数（基础偏移）。
 - KV 缓存读取：与批量大小近似线性正比。
 - 两者之和形成一条向上倾斜的曲线。

总时间 $t = \max(t_compute, t_mem)$ ，取两条曲线的上包络线。

这意味着什么？ 这是一张延迟图。随着批量大小增大，最初延迟对批量大小的依赖较弱，存在一个延迟下界。这已经部分回答了你的问题：**对于给定的硬件配置，延迟存在下界**，即把所有参数从内存读取到芯片所需的最短时间。即便利用全部内存带宽，也无法比这更快。

Dwarkesh： 从你画的斜率来看，如果计算时间的斜率始终高于 KV 缓存对内存时间的贡献斜率，是否意味着批量足够大时，内存永远不是瓶颈？

Reiner： 这对上下文长度非常敏感。随着上下文长度增加，KV 缓存读取时间会不断上升，最终会从计算受限（compute-limited）切换到内存受限（memory-limited）。当两条曲线斜率恰好相等时，意味着系统同时处于内存受限和计算受限的平衡点，这是理想状态。

以一个简单的代数例子说明：假设最优上下文长度是 10 万 Token，如果切换到 20 万 Token，MFU（模型浮点利用率）会降至约 50%。稍微偏离最优区间，对 MFU 的影响是显著的。

Dwarkesh： 稀疏注意力（sparse attention）是否能解决这个问题？

Reiner： 我对稀疏注意力很感兴趣。Dense（密集）注意力的内存读取时间与上下文长度成线性关系，而稀疏注意力的扩展性要好得多。DeepSeek 已经发布了稀疏注意力机制的论文，在 KV 缓存这一项中引入了平方根关系，大幅改善了扩展性。至于各大实验室在实践中用的是什​​么，外部很难确定。

批量大小 vs. 成本（cost per token）图像分析：

成本的含义是：运行这次推理需要占用 GPU 若干毫秒，按小时租用费（例如 2 美元/小时/GPU）换算成成本。而这次推理处理了多少 Token？就是批量大小 B。所以：

每 Token 成本= t/B 每 Token 成本= Bt

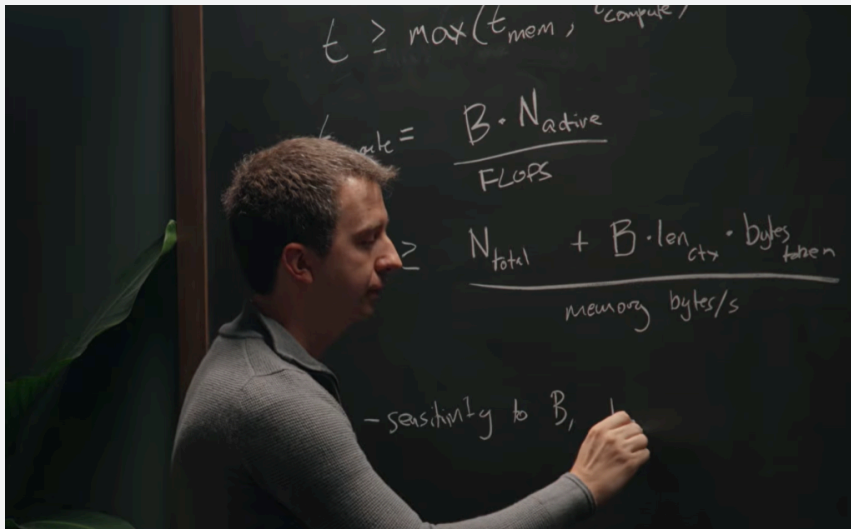
我们把前面三条曲线都除以 B：

- 计算时间曲线：原本与 B 线性正比，除以 B 后变为**常数**。
- KV 缓存读取曲线：原本与 B 线性正比，除以 B 后也变为**常数**。
- 权重读取曲线：原本是常数，除以 B 后变为**双曲线（parabola）**，随 B 增大而下降。

取最大值后，整体形状如下：在批量大小为 1 时，成本极高（权重读取无法被摊销）；随着批量增大，权重读取成本被摊销，趋近于下界，最终由计算时间主导，形成成本下界。



"慢速模式" (Slow Mode) 有没有用？基本没有。因为 KV 缓存和计算对每个批次都是独一无二的，无法通过更大的批量来摊销这两项成本。"慢速模式"只是让请求在这条成本曲线上停留更久，无法突破那条下界。



最优批量大小的计算：

我们关注的是权重读取时间等于权重计算时间的那个点（忽略 KV 缓存项以简化分析）：

$N_{total} \text{ 内存带宽} = B \times N_{active} \text{ FLOPs 内存带宽}$
 $N_{total} = \text{FLOPs} \times B \times N_{active}$

整理后：

$\text{FLOPs} / \text{内存带宽} = B \times (N_{active} / N_{total})$
 $\text{FLOPs} = B \times N_{total} \times N_{active}$

左边是一个硬件参数，称为**算术强度比**。以 FP4 精度为例（每次乘法 0.5 字节），这个比值在大多数 GPU 上约为 **300**（无量纲）。右边的 N_{active} / N_{total} 是稀疏度参数。因此：

$B \geq 300 \times N_{total} \times N_{active} = 300 \times \text{稀疏度}$
 $B \geq 300 \times N_{active} \times N_{total} = \text{稀疏度} \times 300$

以 DeepSeek 为例，激活 256 个专家中的 32 个，稀疏度为 1/8，因此：

$B \approx 300 \times 8 = 2400$
 $B \approx 300 \times 8 = 2400$

这个估算与实践中的数值非常接近。实践中通常会取 2 到 3 倍的余量，因为实际效率不如屋顶线分析理想。所以最优批量大小大约是 **2000 到 3000 个 Token**。

【注：这里的"Token"指的是并发推理序列数——大约 2000 条独立的对话序列同时做单步解码，而非一条长序列中的 Token 数。】

Dwarkesh： 加入 KV 缓存后，最优批量大小会有什么变化？

Reiner： 如果加入 KV 缓存，它会消耗更多内存带宽，权重加载可用的带宽就减少了，因此需要更大的批量来补偿，最优批量大小会增大。

Dwarkesh： 这个数字和 GPU 个数是无关的？

Reiner： 对。结论非常有趣——最优批量大小只取决于稀疏度，与模型规模本身无关（稀疏度本身蕴含了模型规模的信息）。

每秒 Token 数（吞吐量）估算：

每秒 Token 数= $B\Delta t=B\times 64\approx 2000\times 64=128,000$ tokens/s
每秒 Token 数= ΔtB
= $B\times 64\approx 2000\times 64=128,000$ tokens/s

【注： $\Delta t\approx 15\sim 20$ 毫秒，取倒数约为 64/s。】

Dwarkesh： Gemini 去年公布的全球流量是每秒数亿 Token，这只是其千分之一左右。

Reiner： 是的。这说明一个系统至少需要达到 Gemini 千分之一的规模才能在市场上有竞争力。这是一个有意思的下界。

关于稀疏度与模型质量的权衡：

论文《Unified Scaling Laws for Routed Language Models》研究了在保持活跃参数量不变的情况下，增加稀疏度对模型质量的影响。根据旧版 MoE 技术的实验结果，64 个专家、3.7 亿活跃参数的模型，质量与 13 亿参数的 Dense 模型相当。也就是说，总参数量扩大了 64 倍，才换来了相当于 4 倍活跃参数的效果——代价相当大。

Dwarkesh： 稀疏度增大一倍，总参数量就要扩大 8 倍，这到底是合算的吗？

Reiner： 从我们的分析框架来看，这是纯粹的净收益——因为更大的总参数量可以通过更大的批量来摊销，所以只要有足够多的用户，就尽量增加稀疏度。唯一的限制是内存容量：更多的总参数意味着需要更多的内存来存储权重。

Dwarkesh： 关键点是：稀疏度增加，需要的批量也更大，而更大的批量需要更大的内存容量来存储 KV 缓存，这是内存容量而非内存带宽的问题。

Reiner： 完全正确。这是个很好的切入点，下面我们可以来聊聊 MoE 层在 GPU 机架上的物理布局。

第二章：MoE 模型在 GPU 机架上的布局方式

Reiner： 我们先放大看 MoE（混合专家）层的结构。一个典型的 MoE 层包括：

- **路由层（Router）：**接收输入 Token，决定将其路由到哪些专家。
 - **多个专家（Experts）：**路由层选择一小部分专家，例如 256 个中选 1/32。每个专家本身是一个标准 MLP，包含上投影（up projection）、非线性激活和下投影（down projection）。
 - **汇聚与残差连接：**各专家的输出汇聚求和后，加上输入 Token 的残差连接，输出最终结果。
- 如何将 MoE 映射到 GPU 机架？标准做法是使用专家并行（expert parallelism）：不同的专家放在不同的 GPU 上。**

以 DeepSeek 的 256 个专家为例，在 Blackwell 机架的 72 块 GPU 上部署：为简化计算，只用其中 64 块（忽略其余 8 块），每块 GPU 存放 4 个专家。

Token 需要从路由层分发到各个专家所在的 GPU，然后再汇集回来——这产生了**全互联（all-to-all）通信模式**：任意 GPU 都可能向任意其他 GPU 发送数据。

Blackwell 机架内的 NVLink 网络天然支持全互联通信——每块 GPU 通过 NVLink 电缆连接到机架内部的 NVSwitch，任意两块 GPU 只需两跳即可通信（GPU → NVSwitch → GPU）。因此，**单个机架是 MoE 专家并行的完美场景。**

跨机架的问题：

当我需要扩展到两个机架时，麻烦来了。机架间通信使用的是规模扩展网络（scale-out network），其带宽约为机架内 NVLink（scale-up network）的 **1/8**。这意味着：跨机架部署 MoE 时，约有一半的 Token 需要走这条慢速通道，成为严重瓶颈。**因此，单个机架限定了 MoE 专家层的规模上界。**

这也正是行业一直在推动更大互联域（interconnect domain）的动力。

机架的物理结构简介：

机架是一个物理结构，通常高约数米、宽约一到两米，容纳约 64 块 GPU，受限于供电、重量和散热能力。Nvidia 的 Blackwell 机架将 GPU 置于机架外侧，NVSwitch 置于内部，通过大量电缆连接。

- **机架内（scale-up）**：全互联，高带宽，低延迟。
- **机架间（scale-out）**：通过数据中心交换机连接，带宽约为机架内的 1/8。

从 Hopper 到 Blackwell，scale-up 域的规模变化：

- Hopper：8 块 GPU 的 scale-up 域（NVLink 域）
- Blackwell：72 块 GPU（约 64）
- Rubin（下一代）：约 500 块 GPU

从 Hopper 到 Blackwell 主要是从“托盘”形态切换到“机架”形态的产品决策。从 64 到 500 则需要更复杂的物理机架设计，核心挑战是**电缆密度**——随着 GPU 数量翻倍，电缆密度也要翻倍，受限于机架内的物理空间、电缆弯曲半径、背板连接器密度以及重量和散热等多方面约束。

为何不直接建一个超大交换机把所有 GPU 都互联？ 主要原因是布线拥塞——需要铺设的电缆数量极其庞大，物理上难以实现。

更大 scale-up 域对 AI 进展的影响：

GPT-4 据传拥有超过一万亿参数，但直到近半年才有更大规模的模型发布——这是否因为我们一直在等待足够大的内存来容纳一个五万亿参数模型？

Reiner： 是的，这正是关键所在。以 Hopper 为例，8 块 H100 有约 640 GB 显存（截至 2022 年）。而 Blackwell 的 scale-up 内存终于达到 10~20 TB 量级，足以容纳一个五万亿参数模型及其 KV 缓存。更大的 scale-up 域是一次重大解锁。

Google 的 TPU 部署长期拥有较大的 scale-up 域，这也解释了为何 Gemini 似乎在预训练方面领先更早。活跃参数受计算成本限制，总参数受 scale-up 域规模限制——这两者共同界定了可



行的模型设计空间。

第三章：流水线并行如何跨机架分布模型层

Dwarkesh: 我们讨论的单 scale-up 域内操作，是特别适用于某种具体工作负载，还是普遍适用——无论是前向传播还是后向传播，无论是预填充（prefill）还是解码（decode），无论是预训练、RL 生成还是用户推理？

Reiner: 要回答这个问题，我们需要讨论其他通信模式。除了专家并行（all-to-all），还有**张量并行（tensor parallelism）和数据并行（data parallelism），以及流水线并行（pipeline parallelism）**。随着专家粒度越来越细，张量并行已不再那么重要，但流水线并行和数据并行非常适合跨多个机架使用。

流水线并行（Pipeline Parallelism）：

设想我们有一个 MoE 层，上面还有一百多个这样的层。我可以在某一层切换到另一个机架，让不同机架负责不同的层。

关键问题：切换机架会成为通信瓶颈吗？

我们比较 scale-out 带宽需求与 scale-up 带宽需求之比：

$$\frac{t_{scale-up}}{t_{scale-out}} = \frac{18 \times N_{activated\ experts} \times 2 \times N_{layers\ per\ stage}}{t_{scale-out}} = \frac{18 \times N_{activated\ experts} \times 2 \times N_{layers\ per\ stage}}{81 \times N_{activated\ experts} \times 2 \times N_{layers\ per\ stage}}$$

【注：1/8 来自 scale-up 比 scale-out 快 8 倍；×2 来自 all-to-all 的双向通信（上行和下行）；N_activated experts 是每个 Token 激活的专家数；N_layers per stage 是每个流水线阶段的层数。】

我们希望这个比值 ≥ 1，即 scale-up 时间 ≥ scale-out 时间——这意味着 scale-up 不是瓶颈（它速度更快，处理完数据时 scale-out 尚未完成）。

需要克服的只是 8 倍的因子。由于激活专家数通常就在 8 左右，再适当增加每流水线阶段的层数，就能轻松满足这一条件。

实践含义： 可以构建一条由多个机架组成的流水线，每个机架负责几层，然后顺序传递到下一个机架。这种切分方式天然地对应模型架构本身——专家切分在 GPU 之间，层切分在机架之间，非常直观。

Dwarkesh: Ilya 曾说“众所周知，流水线并不明智”，Horace He 也提到流水线会带来架构约束（比如 Kimi 那种跨层残差连接就很难实现）。流水线的好处是什么？

Reiner: 流水线本身带来很大的工程麻烦，但确实有好处：**节省内存容量**。它不降低运行时间或计算量——只是把一部分内存压力从一个机架转移到另一个机架。如果单个机架的内存成为瓶颈，流水线可以大幅缓解这个问题，让模型参数分散在多个机架上存储。

流水线气泡（Pipeline Bubble）与微批次（Micro-batch）：

让我们画出推理时的流水线时序图。假设有 4 个机架（流水线阶段）：



时间 → 机架 1: [批次0][批次1][批次2][批次3][批次0][批次1]... 机架 2: [批次0][批次1][批次2][批次3][批次0]... 机架 3: [批次0][批次1][批次2][批次3]... 机架 4: [批次0][批次1][批次2]...

在**推理**时，我们让批次 0 一进入机架 1，机架 1 就立刻开始处理批次 1——无需等待。这完全填满了时间轴，没有气泡。此时“微批次”和“批次”的区别并无实质意义，只是叫法不同。

在**训练**时，情况更复杂。需要先完成前向传播，再做反向传播，且反向传播需要完整的全局批量才能做权重更新。为了避免气泡，各种方案（如 Zero Bubble、One-Forward-One-Backward）会将前向和反向交织起来，但这带来相当的工程复杂性。

流水线对推理延迟有影响吗？ 没有。延迟与不使用流水线相同——只是把各机架的工作排列在一条时间线上，总时间不变。流水线唯一的好处是降低每个机架的内存容量需求。

Dwarkesh： 那为什么推理时不常用流水线？

Reiner： 因为 Blackwell 机架已经有几十 TB 的内存，而一个万亿参数的模型只需约 1 TB，内存本来就相当富裕，流水线降低的是已经不大的数字，收益有限。

流水线与 KV 缓存的内存分析：

系统内存需求：

$$C_{total} = N_{total} + B \times L_{context} \times bytes_per_token$$

引入专家并行度 E（机架内 GPU 数，例如 64）和流水线并行度 P（机架数，例如 4），每 GPU 内存需求为：

$$C_{per\ GPU} = \frac{N_{total}}{E \times P} + \frac{B_{global} \times L_{context} \times bytes_per_token}{E \times P}$$

但是，引入 P 级流水线时，全局批量 $B_{global} = P \times b_{micro}$ （P 个微批次，每个大小为 b_{micro} ）。代入后：

$$C_{per\ GPU} = \frac{N_{total}}{E \times P} + \frac{b_{micro} \times L_{context} \times bytes_per_token}{E}$$

关键结论：流水线阶段数 P 只能减少权重占用的内存，对 KV 缓存占用的内存没有帮助！ P 的增大使全局批量增大，两个效应恰好抵消。

这类似于之前的结论：KV 缓存无法通过大批量来摊销，现在又发现它也无法通过流水线分担。

Dwarkesh： 所以前沿实验室做推理时，基本上都在单个 scale-up 域内？

Reiner： 是的。对于大多数模型，最优策略是：尽可能多地使用专家并行（最多用满整个 scale-up 域），流水线并行只用极少的级数（0 到 2 级，主要是为了控制权重内存）。张量并行由于专家越来越细，已不再适用。

如果模型极大、极稀疏，超出单个机架的内存，则可以适当增加流水线级数。

更大的 scale-up 域为何重要？

有人会问：既然流水线能解决内存容量问题，更大的 scale-up 域有什么额外价值？



关键在于**内存带宽**，而非内存容量：

$tmem（权重）= N_{total} \times scale-up$ 域内所有 GPU 的总内存带宽 $= N_{total} \times S \times \text{单 GPU 带宽}$
 $tmem（权重）= scale-up$ 域内所有 GPU 的总内存带宽 $N_{total} = S \times \text{单 GPU 带宽}$

【注：S 为 scale-up 域内 GPU 数量。流水线中不同阶段不能并行加载，但同一 scale-up 域内的所有 GPU 可以并行加载各自负责的权重，总带宽是单 GPU 的 S 倍。】

从 Hopper 到 Blackwell，单 GPU 内存带宽提升约 1.5~2 倍，但 scale-up 域大小提升了 8 倍（从 8 到 64），总带宽因此大幅提升。这带来的收益是：

- **更低的推理延迟**；
- **支持更长的上下文**（因为 KV 缓存读取速度更快）——这对日益强调智能体（agentic）能力的模型尤为重要。

第四章：Ilya 为何说"众所周知，流水线并不明智"

Dwarkesh：现在大家都在谈论"内存墙"——内存变得极其昂贵，供应不足。据说超大规模数据中心今年有 50% 的资本开支花在内存上，这意味着消费类设备（手机、笔记本）也受到冲击，产量下降。

但同时，你刚才说 Blackwell 机架内存已经相当富裕。既然流水线能进一步降低内存需求，Jensen Huang 为什么还要把这么多内存堆进这些系统里？

Reiner：让我们来分析内存容量的实际需求。

系统总内存需求：

$C_{total} = N_{total} + B \times L_{context} \times \text{bytes_per_token}$
 $C_{total} = N_{total} + B \times L_{context} \times \text{bytes_per_token}$

流水线可以减少权重部分的需求，但 KV 缓存部分无法被流水线分担。这就是关键所在：当流水线级数 P 足够大，权重项变得微不足道，**KV 缓存成为内存占用的主导项**。

进一步的分析表明：增加流水线级数会相应增加同时在途的序列数（in-flight sequences），两个效应精确抵消，每 GPU 的 KV 缓存内存并不减少。所以，**流水线对于 KV 缓存根本没有帮助**。

Dwarkesh：那推理时实际上用什么并行策略？

Reiner：DeepSeek 的论文里有记载：大量使用专家并行，极少甚至不用流水线（最多用 1~2 级来控制权重存储，不再多了）。张量并行在专家越来越细的今天已几乎没有意义。

为什么超大 Scale-Up 域对 AI 进展如此重要：

总结一下，scale-up 域大小影响 AI 进展的两个核心路径：

- **内存带宽**：更大的 scale-up 域意味着更多 GPU 并行加载权重，总带宽成倍提升，直接降低推理延迟，支持更长上下文。
- **内存容量**：容纳更多总参数、更多 KV 缓存，支持更大规模的模型部署。



流水线解决了内存容量问题（至少对于模型权重），但只有更大的 scale-up 域才能解决内存带宽问题。

第五章：由于强化学习，模型可能比 Chinchilla 最优训练量多 100 倍

Dwarkesh: 现在有了 Chinchilla 扩展律（Chinchilla scaling laws），它告诉你模型大小相对于训练数据量应当如何匹配。但现在的目标不只是用训练算力最大化模型质量，而是最小化训练和推理的综合成本，同时达到某个性能目标。此外，有了强化学习（RL），还要考虑预训练、RL 生成和用户推理这三者之间的计算分配。

具体问题是：现在的模型比 Chinchilla 最优多训练了多少？RL 的引入是否改变了这个数字？

Reiner: 这需要一些推测，因为最新的扩展律和模型流量数据并未公开。但我们可以用一个启发式框架来估算。

基本思路：当总成本是两项成本之和时，最小化总成本的最优点往往在两项成本相等处。 这对形如 $1/x$ 与 x 的函数对成立，对指数函数对也成立，对幂律函数通常也成立。因此，我们的启发式假设是：**预训练成本、RL 成本和推理成本应当大致相等。**

成本公式：

- 预训练计算量（FLOPs）= $6 \times N_{\text{active}} \times D_{\text{pretrain}} \times 6 \times N_{\text{active}} \times D_{\text{pretrain}}$ （著名的 6ND 公式，前向 + 反向 = 6 倍参数乘数据量）

【注：每个参数每个 Token 的前向传播约 2 FLOPs，反向传播约 4 FLOPs，合计约 6 FLOPs。

- RL 计算量 = $\alpha \times N_{\text{active}} \times D_{\text{RL}} \alpha \times N_{\text{active}} \times D_{\text{RL}}$ ，其中 α 在 2~6 之间（2 表示只做生成不做反向传播，6 表示每条轨迹都做完整的前向+反向；实际上还要扣除 decode 的 MFU 低于训练 MFU 的低效因子，约 30%，因此有效 $\alpha \approx 1/10$ ）
- 推理计算量（FLOPs）= $2 \times N_{\text{active}} \times D_{\text{inference}} \times 2 \times N_{\text{active}} \times D_{\text{inference}}$ （只有前向传播，系数为 2）

【注：前向传播 = $2 \times$ 参数量 $\times$ Token 数，这就是推理的 FLOPs 来源。】

令三者相等（系数约 1/10 和 1/10），活跃参数量可约去，得到：

$$D_{\text{pretrain}} \approx D_{\text{inference}} = D_{\text{RL}} \times 110 \quad D_{\text{pretrain}} \approx D_{\text{inference}} \approx D_{\text{RL}} \times 101$$

即：RL Token 数应约为预训练 Token 数的 10 倍（因为 RL 每个 Token 的成本更高，要花同样多的钱就需要更少的 Token）。预训练 Token 数与推理 Token 数大致相当。

实际数值估算：

- 推理 Token 总量：约 5000 万 tokens/秒（假设某单一模型的流量） $\times$ 2 个月 $\approx$ **200 万亿 Token**。
- 前沿模型的预训练 Token 数：据估算约 **150 万亿 Token**（与推理量大致相当，符合我们的框架）。

- 活跃参数量：约 **1000 亿参数**（估算）。
- Chinchilla 最优 Token 数 $D_{Chinchilla} \approx 20 \times N_{active} \approx 2 D_{Chinchilla} \approx 20 \times N_{active} \approx 2$ 万亿 Token。

【注：Chinchilla 规律建议训练 Token 数约为参数量的 20 倍。】

结论： 实际训练 Token 数（约 200 万亿）是 Chinchilla 最优值（约 2 万亿）的 **100 倍**。即当前前沿模型的过训练程度约为 Chinchilla 最优的 100 倍。

Dwarkesh： 这意味着，为了优化训练与推理的综合成本，GPT-5 之类的模型接受用户使用时产生的全部 Token 量，应当与预训练 Token 总量大致相当——而预训练 Token 量大约等于人类知识的总和。

Reiner： 这就是这个框架给出的推论。当然，如果你的模型预测能力不完美，或者模型最终被放弃而没有部署，推理端的 Token 价值要打折扣，因此实际上可能会更倾向于多训练一些。

Dwarkesh： 仅凭公开信息就能首先原理地推算出这种量级的数字，确实令人叹服。下面，我们可以从公开的 API 价格中再推断一些有趣的信息。

第六章：从 API 定价推断长上下文的内存成本

Dwarkesh： Gemini 3.1 Pro 的定价是：超过 20 万 Token 的上下文比 20 万以下贵 50%。为什么恰好是 50%？为什么恰好在 20 万 Token 这个节点？

Reiner： 先回顾一下成本与上下文长度的关系图。以上下文长度为横轴，每 Token 成本为纵轴：

- **计算时间（compute time）：** 对上下文长度几乎无依赖，是一条水平线。（理论上存在二次项，但在百万 Token 量级以下可以忽略。）
- **内存读取时间（mem time）：** 从权重基础值出发，随上下文长度线性增加（因为 KV 缓存随上下文增大）。

两者取最大值，在某个临界点会从“计算受限”切换到“内存受限”，出现拐点。**这个拐点大致对应提价的 20 万 Token 节点。** 两段式定价结构（低于 20 万一个价，高于 20 万一个价）是应对这一成本结构的合理商业策略。

从定价推算 bytes_per_token（每 Token 的 KV 缓存大小）：

令内存时间等于计算时间的断点在 200K Token 处（忽略权重读取项，仅考虑 KV 缓存读取项）：

$$B \times L_{context} \times \text{bytes_per_token} \text{内存带宽} = N_{active} \text{FLOPs内存带宽} \\ B \times L_{context} \times \text{bytes_per_token} = \text{FLOPs} N_{active}$$

B 约去，整理得：

$$\text{bytes_per_token} = N_{active} L_{context} \times \text{内存带宽} \\ \text{FLOPs} = N_{active} L_{context} \times 1300 \text{bytes_per_token} = L_{context} N_{active} \times \text{FLOPs内存带宽} = L_{context} N_{active} \times 3001$$



代入 $N_{active}=1000$ $N_{active}=1000$ 亿, $L_{context}=200,000$ $L_{context}=200,000$:

$bytes_per_token=10112 \times 105 \times 1300 \approx 1066 \approx 1667$ 字节 ≈ 2 KB
 $bytes_per_token=2 \times 1051011 \times 3001 \approx 6106 \approx 1667$ 字节 ≈ 2 KB

2 KB/token 是否合理? 完全合理。可以通过以下两条路径实现:

- **密集注意力 + 跨层共享:** 如 [Character.AI](#) 和 Gemma 模型中的架构, 全局 KV 缓存只有 1 层, 共享给所有层使用。计算: $1 \times 2 \times d_{head} \times N_{KV} \text{ heads} = 1 \times 2 \times 128 \times 8 = 2048$
 $1 \times 2 \times d_{head} \times N_{KV} \text{ heads} = 1 \times 2 \times 128 \times 8 = 2048$ 字节。
- 其中 $d_{head}=128$ ($d_{head}=128$ (注意力头维度, 典型值)); $N_{KV} \text{ heads}$ 通常在 1~8 之间。
- KV 头 (存储历史 Token 表示, 留在内存中) 与 Q 头 (只在当前 Token 的注意力计算中临时使用) 不同。
- **稀疏注意力:** 使用更多层和更多头, 但引入一个稀疏因子 ($1/sparsity$) 来降低等效的 $bytes_per_token$ 。

这进一步说明, API 定价实际上泄露了大量模型架构信息。

从输出价格比输入价格贵推断 decode vs. prefill 的成本差异:

通常输出 (decode) 的价格比输入 (prefill) 贵约 5 倍。为什么?

我们画 "pass 长度 (len_pass) vs. 每 Token 成本" 的关系图:

- **decode 是 $len_pass = 1$ 的特殊情况。**
- **prefill 对应较大的 len_pass 。**

每 Token 成本 = t / len_pass :

- **计算成本 ($t_{compute} / len_pass$):** 计算时间本身不随 len_pass 变化, 除以 len_pass 后是一条常数线——这意味着 prefill 的每 Token 计算成本与 decode 相同。
- **内存成本 (t_{mem} / len_pass):** 内存时间随 len_pass 的增加而...其实几乎不变 (权重读取是主要项, KV 缓存读取在 flash attention 下几乎是临时的)。但除以 len_pass 之后, 反而随 len_pass 增大而降低。

这说明: **prefill 实际上比 decode 便宜, 因为 decode 极度受限于内存带宽, 而 prefill 可以更高效地利用计算能力。** decode 是内存带宽受限的, prefill 是计算受限的。

从 "output 比 input 贵 5 倍" 这一定价, 可以读出: decode 时内存带宽利用率约是计算利用率的 5 倍——即系统极度受内存带宽瓶颈制约。

提示词缓存 (Prompt Cache) 的定价分析:

以 Gemini 2.5 Pro 的定价为例 (非精确):

- 基础输入 Token: \$5/百万 Token (相当于重新计算 KV 缓存的成本)

- 写入缓存（5 分钟）：略贵于基础价格

- 写入缓存（1 小时）：更贵

缓存的成本有两个维度：

- **检索成本（一次性）**：从存储位置读取 KV 缓存到 HBM 的带宽成本。

- **持有成本（每秒）**：占用存储空间的机会成本（若占满该存储，GPU 无法处理更多请求）。

不同内存层级的“排空时间”（capacity / bandwidth）：

- **HBM**：≈ 20 毫秒（排空时间极短，不适合长时间持有）

- **DDR**：≈ 秒级（1~10 秒）

- **Flash（NVMe SSD）**：≈ 分钟级（约 1 分钟）

- **机械硬盘（HDD）**：≈ 小时级（约 1 小时）

5 分钟缓存 vs. 1 小时缓存恰好对应 Flash 和 HDD 两个层级。 令人意外的是，机械硬盘这种古老技术仍在数据中心中被使用，其排空时间约为 1 小时，成本极低但速度极慢。

第七章：神经网络与密码学的趋同演化

Dwarkesh：你有一篇非常有趣的博文，讨论了密码协议的结构与神经网络的相似性——两者都需要将信息混合到所有输入中（前者是为了防止哈希函数被预测，后者是为了建模输入之间的相互影响），这是一种趋同演化。但从高层次看，它们其实在做相反的事情：密码协议把有结构的信息变得像随机数，神经网络则从看似随机的数据（蛋白质序列、DNA、自然语言）中提取高层结构。

Reiner：是的，这个对比很有意思。相似机制用于相反目的。我们也能在其他地方看到“混合与扰乱”的模式，比如做蛋糕时搅拌面糊——先这个方向搅，再那个方向搅，确实是不错的混合策略。

不过，两者有一个深刻的区别：神经网络是可微分的，而密码算法努力避免可微分。

- **可微分性使神经网络可训练。** 残差连接和 LayerNorm 等设计都是为了保持梯度的简洁可计算性。
- **密码分析中的差分密码分析（differential cryptanalysis）** 恰恰是通过对密码算法“求导”来攻击它：对输入做微小扰动，观察输出变化。一个好的密码算法应该使得输入的微小差异导致输出的巨大差异（雪崩效应），而神经网络恰恰需要保持梯度的连续性来避免雪崩。

两者的目标在这一维度上截然相反。

Dwarkesh：神经网络真的被用于密码学了吗？

Reiner：用神经网络来做密码算法是非常危险的。99% 的新密码算法都是被攻破的。

但反方向——密码学的思想被引入神经网络——至少有一个非常成功的例子：**Feistel 密码（Feistel Cipher / Feistel Network）**。

Feistel 网络原理：给定一个不可逆函数 f，如何构造一个可逆层？方法是使用两个输入：



输入: $(x, y) \rightarrow$ 输出: $(x, y+f(x))$ 输入: $(x, y) \rightarrow$ 输出: $(x, y+f(x))$

- **加密（前向）**：计算 $z=y+f(x)$ $z=y+f(x)$ ，输出 (x, z) (x, z) 。
- **解密（逆向）**：已知 (x, z) (x, z) ，恢复 x （直接读取），恢复 $y=z-f(x)$ $y=z-f(x)$ （已知 x ，可以重新计算 $f(x)$ ）。

整个构造是可逆的，即使 f 本身不可逆。这在密码学中被广泛用于构建加密层，也是许多对称加密算法的基础。

被引入神经网络的应用——可逆网络（RevNets）：

2017 年的论文《Reversible Residual Networks》（RevNets）将 Feistel 思想引入 Transformer 等神经网络：

两个输入： (x, y) 网络层 f （例如 Transformer 层） 前向： $output_x = x$
 $output_y = y + f(x)$ 逆向： $x = output_x$ $y = output_y - f(output_x)$

这实际上是将残差连接从 1 层变成了跨 2 层的连接（ y 来自上一层的残差）。

好处：彻底消除激活值内存占用。

- **普通训练**：前向传播时需要将每一层的激活值写入 HBM，反向传播时再读出（内存占用随层数线性增加，往往是训练中最大的内存开销）。
- **RevNets 训练**：因为网络可逆，前向传播时可以不保存激活值；反向传播时，同步地从前向传播的最终状态逆向重构出所需的激活值（重算，rematerialization）。

代价是：需要额外的计算（重算一遍前向传播），换来了大幅减少的内存占用。

Dwarkesh：有趣——这和 KV 缓存的逻辑正好相反：KV 缓存是用更多内存来节省计算，而 RevNets 是用更多计算来节省内存。

Reiner：完全正确。鉴于当前硬件的内存与计算成本比，“花内存省计算”（如 KV 缓存）通常是更合算的；但 RevNets 展示了反过来也可以有价值。

Dwarkesh：太精彩了，Reiner，非常感谢你！这场黑板讲座完全实现了我们建造这个新录制空间的初衷。

Reiner：非常感谢，很高兴能来！

视频地址：<https://www.youtube.com/watch?v=xmkSf5IS-zw>

风险提示及免责条款

市场有风险，投资需谨慎。本文不构成个人投资建议，也未考虑到个别用户特殊的投资目标、财务状况或需要。用户应考虑本文中的任何意见、观点或结论是否符合其特定状况。据此投资，责任自负。

👍 182 ★ 收藏

分享到:  

写评论

请发表您的评论





表情



图片

发布评论

